

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand
9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply

10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply
----	--	-------------

Code of Conduct:

I Dheeraj Krushna P / 192525230 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P. DheerajKrushna

Q1. CREATE TABLE — syntax errors and missing constraints (BL2)

Faulty statement:

```
CREATE TABLE Student (  
    StudentID INT,  
    Name VARCHAR(50)  
    Age INT,  
    Dept VARCHAR(20),  
    PRIMARY KEY StudentID  
);
```

Errors identified:

- Missing comma between the Name and Age column definitions.
- PRIMARY KEY StudentID is invalid syntax — the column must be wrapped in parentheses.
- No NOT NULL on Name, so a student record could be inserted with no name.
- No CHECK constraint on Age, so a negative or unrealistic age could be stored.

Corrected statement:

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Age INT CHECK (Age > 0),  
    Dept VARCHAR(20)  
);
```

Explanation: the missing comma was a plain syntax error that would stop the statement from parsing at all. PRIMARY KEY(StudentID) (or the inline form used here) is required syntax to designate the key. Adding NOT NULL and CHECK moves basic data-quality rules into the schema itself instead of relying on the application layer to enforce them.

Q2. Aggregate function used without GROUP BY (BL2)

Faulty query — intended to show each department's average salary:

```
SELECT Dept, AVG(Salary)  
FROM Employee;
```

Error identified: Dept is a non-aggregated column selected alongside an aggregate function (AVG) with no GROUP BY clause. Most RDBMSs (Postgres, SQL Server strict mode) reject this outright; MySQL in non-strict mode silently returns one arbitrary Dept value paired with the overall average, which is misleading rather than an average per department.

Corrected query:

```
SELECT Dept, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY Dept;
```

Explanation: GROUP BY Dept partitions the rows by department first, so AVG(Salary) is computed once per partition, giving one correct average per department rather than a single average across the whole table.

Q3. Misspelled / improperly referenced columns (BL1)

Faulty query:

```
SELECT StudntName, Departmnt
FROM Studnet
WHERE Age > 20;
```

Errors identified: the table name Studnet is misspelled (should be Student); the columns StudntName and Departmnt are misspelled (should be StudentName and Department). Running this as written raises an "unknown table/column" error.

Corrected query:

```
SELECT StudentName, Department
FROM Student
WHERE Age > 20;
```

Explanation: SQL identifiers must exactly match the names defined in the schema (case-sensitivity depends on the RDBMS and collation, but spelling never does); checking the CREATE TABLE definition against the query is the standard first debugging step for "column/table not found" errors.

Q4. JOIN with a missing or incorrect join condition (BL3)

Faulty query — intended to list each student with their enrolled course title:

```
SELECT S.Name, C.CTitle
FROM Student S
JOIN Enrollment E
JOIN Course C ON E.CourseID = C.CourseID;
```

Errors identified: the join between Student and Enrollment has no ON condition at all, so the parser either rejects the statement or (if written as a comma join) silently produces a Cartesian product — every student paired with every enrollment row, most of them meaningless.

Corrected query:

```
SELECT S.Name, C.CTitle
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID;
```

Explanation: every JOIN needs its own ON clause naming the foreign-key relationship it relies on. Adding ON S.StudentID = E.StudentID restricts the join to only the enrollment rows that actually

belong to each student, eliminating the Cartesian product and giving the intended one-row-per-actual-enrollment result.

Q5. Subquery returning multiple values used with '=' (BL2)

Faulty query — intended to find students enrolled in any CSE course:

```
SELECT Name
FROM Student
WHERE StudentID = (
    SELECT StudentID FROM Enrollment
    WHERE CourseID IN (SELECT CourseID FROM Course WHERE Dept = 'CSE')
);
```

Error identified: the '=' operator expects the subquery to return exactly one value, but the inner subquery can return many StudentIDs (many students take CSE courses). This raises a "subquery returns more than one row" error at run time.

Corrected query:

```
SELECT Name
FROM Student
WHERE StudentID IN (
    SELECT StudentID FROM Enrollment
    WHERE CourseID IN (SELECT CourseID FROM Course WHERE Dept = 'CSE')
);
```

Explanation: replacing '=' with IN allows the outer WHERE clause to match against a whole set of values instead of exactly one, which correctly expresses "StudentID is one of these many matching IDs." '=' should only be used when the subquery is guaranteed (e.g., by a unique/primary key filter) to return a single row.

Q6. VIEW with invalid column references (BL1)

Faulty statement:

```
CREATE VIEW StudentGrades AS
SELECT Name, Grade, CourseName
FROM Student, Enrollment;
```

Errors identified: CourseName does not exist in either Student or Enrollment (the course title lives in the Course table, as CTitle) — this is an invalid column reference. The FROM clause also lists two tables with no join condition (an implicit Cartesian product), and Grade exists only in Enrollment while Name exists only in Student, so without a join predicate the pairing is meaningless.

Corrected statement:

```
CREATE VIEW StudentGrades AS
SELECT S.Name, E.Grade, C.CTitle AS CourseName
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID;
```

Explanation: every column referenced in a view must exist in one of the FROM tables (or be an alias), and rows across multiple base tables must be combined with an explicit join condition; the corrected view aliases C.CTitle to the friendlier CourseName so the view's exposed column name matches what the original definition intended.

Q7. Unnecessary conditions — simplifying the WHERE clause (BL3)

Original query with redundant conditions:

```
SELECT Name
FROM Student
WHERE Age > 18
      AND Age > 15
      AND Dept = 'CSE'
      AND (Dept = 'CSE' OR Dept = 'CSE');
```

Issues identified: Age > 15 is redundant once Age > 18 is already required (the second is strictly weaker and adds nothing). The OR clause (Dept = 'CSE' OR Dept = 'CSE') is a duplicate of itself and duplicates the earlier Dept = 'CSE' condition too. None of this changes the result, but it forces the optimizer to evaluate extra predicates on every row.

Optimized query:

```
SELECT Name
FROM Student
WHERE Age > 18
      AND Dept = 'CSE';
```

Explanation: simplifying to the tightest, non-redundant condition set produces an identical result set with fewer predicate evaluations per row, and a simpler WHERE clause is also easier for the query optimizer to match against an index on (Dept, Age).

Q8. Integrity constraint violations in given data (BL2)

Table as populated, with schema PRIMARY KEY(AccNo), FOREIGN KEY(CustID) REFERENCES Customer(CustID), CHECK (Balance >= 0):

AccNo	CustID	Balance	
A001	CU1	5000	
A001	CU2	3000	<- duplicate AccNo (violates PRIMARY KEY / entity integrity)
A002	CU9	2000	<- CU9 does not exist in Customer (violates FOREIGN KEY / referential integrity)
A003	CU1	-500	<- negative balance (violates CHECK constraint)

Violations identified:

- Row 2 repeats AccNo A001, breaking the PRIMARY KEY's uniqueness requirement — two different accounts cannot share the same account number.

- Row 3 references CustID CU9, which has no matching row in Customer, breaking referential integrity — an account cannot belong to a non-existent customer.
- Row 4 has Balance = -500, breaking the CHECK (Balance >= 0) constraint — a balance cannot be negative under this business rule.

Corrections:

```
UPDATE Account SET AccNo = 'A004' WHERE AccNo = 'A001' AND CustID = 'CU2';
DELETE FROM Account WHERE CustID NOT IN (SELECT CustID FROM Customer);
UPDATE Account SET Balance = 0 WHERE Balance < 0;
```

Explanation: each fix targets the specific constraint that was broken — reassigning a unique account number restores entity integrity, removing (or reassigning) rows with a non-existent CustID restores referential integrity, and clamping negative balances restores the domain rule; going forward, the CREATE TABLE constraints (already present in the schema) prevent these rows from being inserted again.

Q9. Relation violating 1NF / 2NF (BL3)

Relation violating 1NF — repeating group in a single column:

```
StudentCourses(StudentID, Name, Courses)
```

StudentID	Name	Courses
S1	Arjun	DBMS, OS, Networks

← multiple values in one cell

Fix for 1NF — split the repeating group into one row per value:

```
StudentCourses(StudentID, Name, CourseName)
```

StudentID	Name	CourseName
S1	Arjun	DBMS
S1	Arjun	OS
S1	Arjun	Networks

This relation is now in 1NF but the key is (StudentID, CourseName); Name depends only on StudentID, a partial dependency on part of the composite key — a 2NF violation.

Fix for 2NF — remove the partial dependency by splitting off Name:

```
Student(StudentID, Name)
Enrollment(StudentID, CourseName)
```

Explanation: 1NF requires every cell to hold a single atomic value, so the comma-separated Courses list had to be split into separate rows. 2NF then requires every non-key attribute to depend on the whole composite key, not just part of it — since Name depended only on StudentID (not on CourseName too), it was moved into its own Student table, leaving Enrollment holding only the fact that genuinely needs the composite key.

Q10. Redundant attributes — normalization to 3NF (BL3)

Relation with redundant, transitively dependent attributes:

```
Employee(EmpID, EmpName, DeptID, DeptName, DeptLocation)
```

Redundancy identified: DeptName and DeptLocation repeat identically on every employee row belonging to the same department. The key is EmpID; DeptName and DeptLocation depend on DeptID, and DeptID depends on EmpID — so DeptName and DeptLocation depend on EmpID only transitively (through DeptID). This is a 3NF violation (a transitive dependency on the key) and means the department's location must be updated in every employee row if it changes.

Normalized to 3NF:

```
Employee(EmpID, EmpName, DeptID)  
Department(DeptID, DeptName, DeptLocation)
```

Explanation: moving DeptName and DeptLocation into their own Department table, keyed by DeptID, removes the transitive dependency — every non-key attribute in each table now depends directly and only on that table's own primary key. Department details are now stored once per department instead of once per employee, eliminating the update anomaly.